Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Масштабируемый и эффективный подход к решению задачи обратного геокодирования на базе Uber H3 / Scalable and efficient approach for solving reverse geocoding problem based on Uber H3**

Работу выполнил / Thesis is executed by	**Гариханов Айдар Дамирович / Aidar Garikhanov**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Маццара Мануэль / Manuel Mazzara**	подпись / signature

Иннополис, Innopolis, 2022

Contents

1 Introduction **7**

2 Literature Review **10**

2.1 Reverse geocoding 10

2.1.1 Applications of geocoding 10

2.1.2 Types of geocoding 11

2.2 Discrete global grid systems 12

2.3 Conclusion 13

3 Methodology **14**

3.1 Grid System 14

3.2 Data model 15

3.3 Dataset 19

3.4 Database 20

4 Implementation **22**

4.1 Processing dataset 23

4.1.1 Data extraction 23

4.1.2 Tiling areas with hexagons 28

4.2 Populating Database 32

4.3 Querying database 35
4.4 Demo application 35

5 Evaluation and Discussion 37

5.1 Algorithm evaluation 37
    5.1.1 Correctness 37
    5.1.2 Performance 40
    5.1.3 Conclusion 42

6 Conclusion 43

Bibliography cited 45

List of Tables

I Format of DBF files 24

II H3 Core Library overview 28

III H3 resolution table [18] 31

IV Format of CSV files 38

V Accuracy of the reverse geocoding algorithm. 39

VI Measurements of the time required to tile all countries and their subdivision by hexagons of different resolutions. 40

List of Figures

1.1 Reverse geocoding using Google Geocoding API 8

1.2 H3: Uber's Hexagonal Hierarchical Spatial Index 8

3.1 Data model with one table 15

3.2 Data model with separate tables 16

3.3 Data model with separate tables (denormalized). 17

3.4 Data model with separate tables (normalized). 18

3.5 Search result from OpenStreetMap. (term: “Kazan“) 20

4.1 Data preparation for the reverse geocoding algorithm. 22

4.2 Region of Kazan tiled with hexagons of resolution 7. 29

4.3 A boundary of Kazan tiled with hexagons of resolution 7. . . . 30

4.4 Uncovered areas inside the region after the compaction. 32

4.5 Data model in Scylla. 33

4.6 Web application which uses the reverse geocoding algorithm. . . 36

5.1 The point is located between two islands of American Samoa. . 39

5.2 Dependency of the filling time on the number of filled indexes. . 41

Abstract

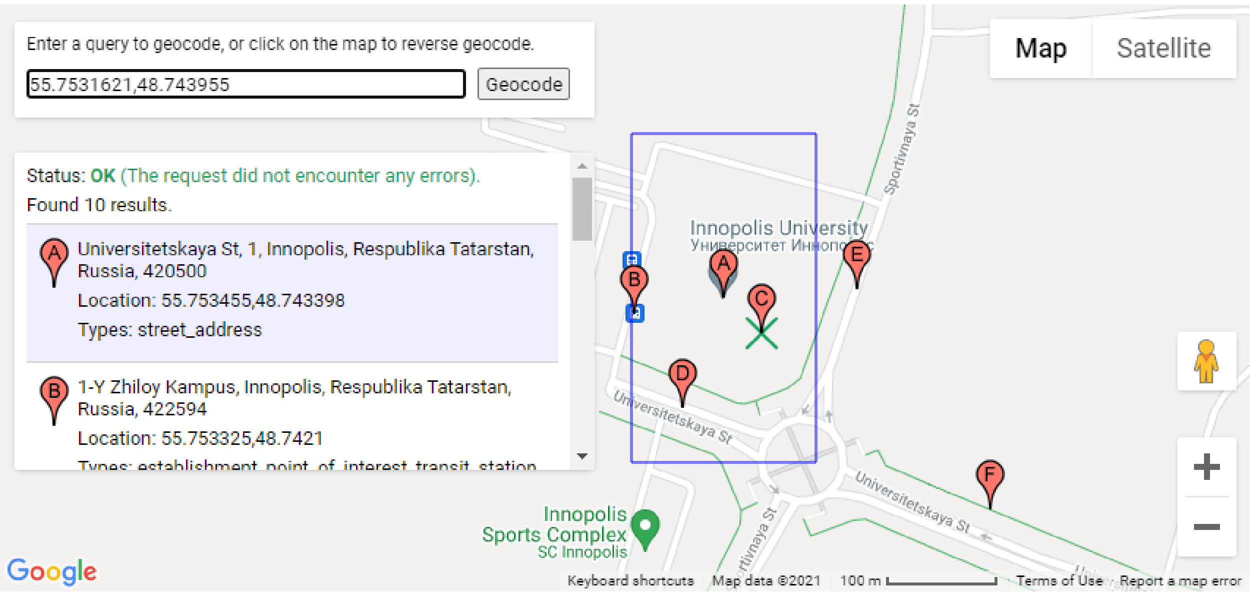
Reverse geocoding is the process of mapping coordinates to a human-readable address. Modern data intensive applications might require highly scalable and efficient reverse geocoding algorithms. Conventional geocoding tools are difficult to integrate with software systems, while online geocoding services provide only limited throughput. This study describes the implementation of a reverse geocoding algorithm based on Uber H3. Both scalability and efficiency are achieved by Scylla - NoSQL distributed database. The implemented algorithm reached a throughput of thousands requests per second with the capability of system scaling.

Chapter 1

Introduction

Geocoding and reverse geocoding occupy a prominent place in modern web applications and services. According to [1], geocoding can be described as the process of converting human-readable addresses into a pair of geographic coordinates.

Reverse geocoding is the opposite process that maps latitude and longitude to a location. A location might be represented as a postal address or the name of the place. For example, Google Maps Geocoding API [2] converts coordinates (55.7531621,48.743955) to the address “Universitetskaya St, 1, Innopolis, Respublika Tatarstan, Russia, 420500” as illustrated in Fig. 1.1. Further in this chapter, mainly the term "geocoding" will be used, but it might refer to reverse geocoding as well.



Enter a query to geocode, or click on the map to reverse geocode.

55.7531621,48.743955

Status: OK (The request did not encounter any errors).

Found 10 results.

Universitetskaya St, 1, Innopolis, Respublika Tatarstan, Russia, 420500

Location: 55.753455,48.743398

Types: street_address

1-Y Zhiloy Kampus, Innopolis, Respublika Tatarstan, Russia, 422594

Location: 55.753325,48.7421

Types: establishment, point_of_interest, transit_station

Geocode
A
B
Map
Satellite
Sportivnaya St
Innopolis University
A
B
C
D
E
F
Universitetskaya St
Universitetskaya St
Sportivnaya St
Innopolis
Sports Complex
SC Innopolis
Google
Keyboard shortcuts
Map data ©2021
100 m
Terms of Use
Report a map error

Fig. 1.1. Reverse geocoding using Google Geocoding API

Uber developed a grid system called “Hexagonal Hierarchical Spatial Index” (H3), which divides the Earth into grid cells of 16 different resolutions [3]. As seen in Fig. 1.2, each grid cell represents a hexagon which is tiled with seven smaller hexagons.

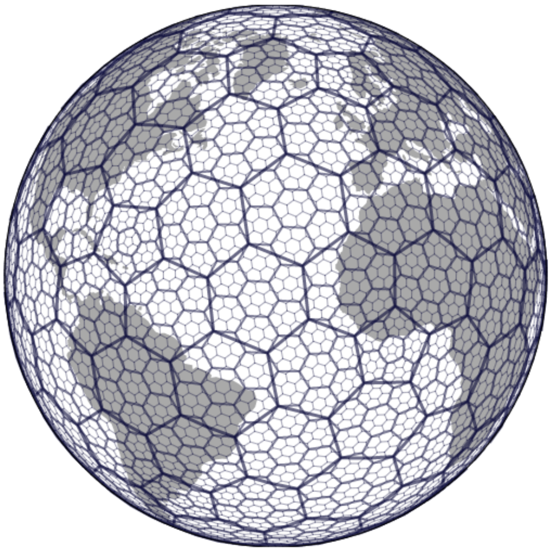


Fig. 1.2. H3: Uber’s Hexagonal Hierarchical Spatial Index

Uber uses H3 to optimize their marketplace for users, in particular to avoid the lack of cars in high-demand areas. However, H3 can also be used for traffic analysis, geographic visualization, and geospatial sharding. This thesis describes how geohash indexing based on H3 helps process a large amount of data by reverse geocoding.

Geocoding algorithms are usually divided into two categories: conventional and online. Conventional reverse geocoding runs on a computer with the reference database prepared. On the other side online geocoding allows users to reverse-geocode coordinates by making web requests. In cases when a software system must aggregate thousands of locations in real-time by geographical regions, conventional geocoding algorithms might be difficult to set up and integrate with the system. At the same time, each request to online geocoding services is charged, and it might too expensive to process large amount of coordinates. In such situations, a different approach for the algorithm is required. This study describes how to implement an efficient and scalable reverse geocoding algorithm based on Uber H3.

Chapter 2

Literature Review

2.1 Reverse geocoding

2.1.1 Applications of geocoding

Both geocoding and reverse geocoding have experienced a rapid growth in diverse areas. For instance, in public health, geocoding has been used to analyze the spatial distribution of cancer based on diagnosis addresses mapped to geographic coordinates [4]–[6]. Survey statistics apply a similar technique to geocode addresses of survey respondents and relate spatial data to survey data [7], [8]. Apart from this, Andresen *et al.* [9] used geocoding for mapping addresses with crime scenes to corresponding coordinates. They analyzed the retrieved data to find spatial patterns of different crime types.

Having the ability to map addresses to geographic coordinates and vice versa is equally important in business. The article by Informatica LLC [10] describes how entrepreneurs can benefit from GIS technologies and establish their workflow processes more effectively. Restaurants, logistics, delivery, and financial services, as well as in marketing, healthcare, and insurance actively use geocoding. For instance, the following use case demonstrates how a delivery company might benefit by using reverse geocoding. Many people use smartphones with GPS modules that can calculate geographic coordinates. A customer, willing to order delivery, opens a mobile application and makes an order sending his or her geographic coordinates. The pair of latitude and longitude is converted to the street address using reverse geocoding. A courier receives the address and performs the order. Ramella [11] compared five enterprise software tools for geocoding. They all have common geocoding functionality, but some services provide distinctive features such as address spell check, map visualization, and packages for mobile devices.

2.1.2 Types of geocoding

In his study, Li [1] divides geocoding and reverse geocoding into two main categories: conventional and online. Conventional geocoding is performed locally on a computer with all the necessary geographic information systems (GIS) tools installed.

According to [1] and [12] a conventional geocoding system consists of four main components: input data, reference dataset, processing algorithm, and output data. The processing algorithm may vary among various systems, but usually, it consists of the following steps:

- Input data normalization. The algorithm separates an input address into tokens and removes all punctuation.

- Standardization. In this step, all extra information is removed from an address so that all data entries follow the same format.

- Address matching. The parts of standardized input are used to find appropriate addresses in the reference dataset. Additionally, attribute weighting is used to determine the relevance of the matches.

Consequently, conventional geocoding tools require users to understand how to use and configure them. Moreover, users should supply a large reference dataset with reference points as input. It is a perfect choice for professionals focusing on GIS technologies, but it might be a complicated process for most business applications.

On the other hand, online geocoding services make integrating ready-made solutions in almost any system easy. Such services provide a platform and programming language independent application programming interface (API) which allows users to make web requests to geocode or reverse geocode provided geographic locations. Asavasuthirakul and Karimi [13] compared and evaluated five online geocoding and reverse geocoding services: MapQuest, Geocoder.us, Google Maps, Yahoo!, and MapPoint. All of them provide an API, but they have different request rates and API interfaces. Two primary metrics for comparing geocoding services are positional accuracy and match rate. Positional accuracy represents a distance between the real and geocoded locations. Match rate is a ratio of correctly geocoded locations.

2.2 Discrete global grid systems

Sahr *et al.* [14] describe Discrete Global Grid (DGG) as “a set of regions that form a partition of the Earth’s surface, where each region has a single point contained in the region associated with it”. In such grids, each point or region is represented as a cell. A Discrete Global Grid System (DGGS) consists of the DGGs with a monotonically increasing number of cells. Bousquin [15] compared two popular open-source implementations of DGGSs: H3 and dggridR. Both of them use hexagons to represent cells. They are also based on icosahedral map projection with a pentagon at each vertex of the icosahedron (polyhedron with 20 faces). After the comparison, dggridR was more flexible in terms of scaling and using smaller units. On the other hand, H3 performed better, locating neighbors and switching scales more efficiently. In addition, H3 was able to aggregate points, lines, and grids into hexagons.

It is worth mentioning, Google developed their own open-source, hierarchical DGGS called S2 [16]. The main difference is that S2 uses square cells while H3 uses hexagons which leads to the following effects [17]:

- Neighbors. Square cells have two types of neighbors: adjacent by an edge or sharing the same point. Hexagons have only one type of neighbor: adjacent by an edge.
- Subdivision. Square cells are subdivided into four smaller squares, while hexagonal cells are tiled with seven smaller hexagons.
- Distortion. While S2 squares are often distorted to a quadrilateral, H3 hexagons stay almost unaltered.

2.3 Conclusion

Overall, the choice of the geocoding method depends on the task that needs to be solved. For general purpose, an online geocoding service is an optimal choice as it is easy to use; however, in some situations, it might be essential to change the reference dataset or process large datasets via batch geocoding. Usually, in such scenarios, conventional geocoding is the only choice.

Chapter 3

Methodology

In this study, I design a reverse geocoding, which enables users to build a reference dataset from scratch and extract relevant region features from a large number of geographic coordinates. The method is based on Uber's H3 geospatial indexing system.

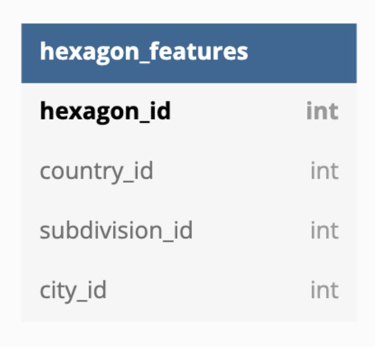
Firstly, Section 3.1 explains what discrete global grid system suits the algorithm the most. Further, Section 3.2 describes the data models which can be used to represent relations between geographic features and H3 indexes. Meanwhile, Section 3.3 reviews public datasets providing reference geographical data. Lastly, Section 3.4 compares a few well-known database management systems by correspondence to the data model, performance, and scalability.

3.1 Grid System

There are several reasons why I chose Uber H3 as a primary grid system. First, each hexagonal cell has its own unique integer 64-bit index. It makes it simple to store spatial data in database management systems, as this ID can be used as a key in both relational and key-value databases. Second, H3 has different 16 resolutions for hexagonal cells [18]. The largest cells are of resolution 0, while the smallest - 15. Users can choose a resolution depending on the desired accuracy. For example, there are approximately 691 thousand hexagons of resolution 8 with an average hexagon area of 0.74 squared kilometers. Third, Uber H3 provides a core library written in C with binding for many popular programming languages, including Python, which I primarily use.

3.2 Data model

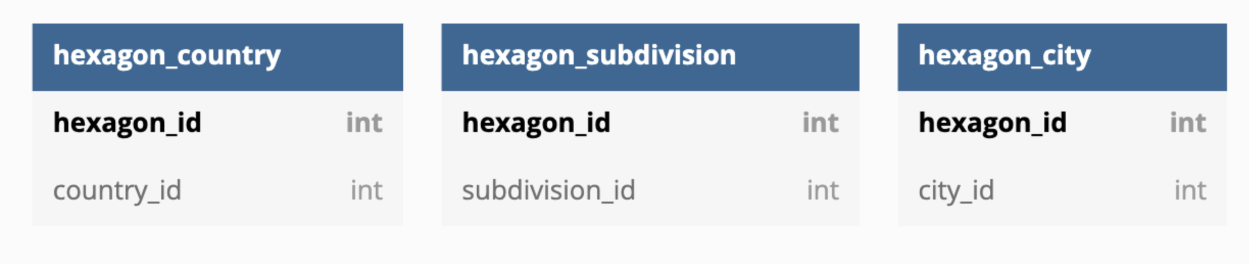
The data model should describe linking a geographic location to specific features representing points of interest (POI). In particular, IDs of H3 hexagons can be used to represent geographic locations, while countries, their subdivisions, and cities represent POIs. Later on, by subdivisions, I will refer to administrative divisions - the largest administrative units of a country (states, regions, counties, provinces, etc.). Although I will implement the reverse geocoding algorithm only for countries and subdivisions, I also include cities in the data models. Fig. 3.1 shows a simplified data model which can be used in both relational and key-value databases.



hexagon_features	
hexagon_id	**int**
country_id	int
subdivision_id	int
city_id	int

Fig. 3.1. Data model with one table

In this case, users query the data from only one table. It is more appropriate if all POIs (cities, countries, etc.) are indexed with hexagons of the same resolution. However, if resolutions of H3 hexagons are different for any of these geographic features, it makes sense to store them in separate tables,x as shown in Fig. 3.2.



hexagon_country	
hexagon_id	int
country_id	int

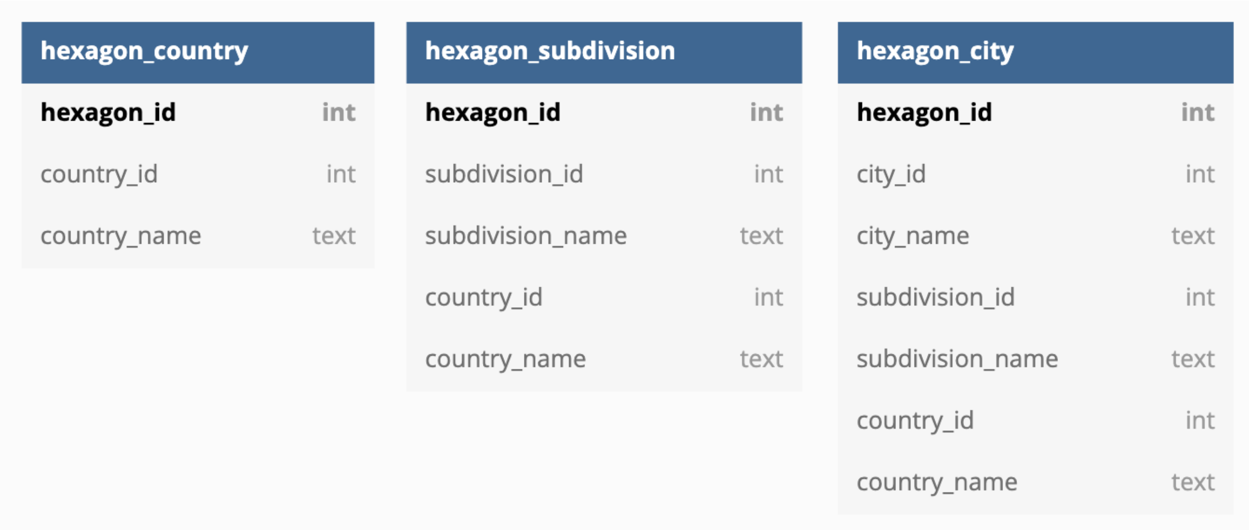
hexagon_subdivision	
hexagon_id	int
subdivision_id	int

hexagon_city	
hexagon_id	int
city_id	int

Fig. 3.2. Data model with separate tables

Although for a system designer, it is more convenient to operate on IDs of POIs, end-users of the application would like to see something more meaningful than just numerical identifiers. Most likely, the database will include more information about POIs (e.g., names of geographic features). In addition, it can be worth considering the inclusion relation between cities, subdivisions, and countries.

There are two basic approaches to organize data with additional information about POIs and relations between them. The first one is to add additional columns in every table which contains IDs of POIs. As shown in Fig. 3.3, this leads to data denormalization.



hexagon_country	
hexagon_id	int
country_id	int
country_name	text

hexagon_subdivision	
hexagon_id	int
subdivision_id	int
subdivision_name	text
country_id	int
country_name	text

hexagon_city	
hexagon_id	int
city_id	int
city_name	text
subdivision_id	int
subdivision_name	text
country_id	int
country_name	text

Fig. 3.3. Data model with separate tables (denormalized).

The second approach is to create separate tables for additional data and relations. Fig. 3.4 shows three new tables for additional data (`city`, `subdivision`, `country`) and two tables for relations (`city_subdivision`, `subdivision_country`). The database schema meets the third normal form because the columns of every table depend only on the primary key.

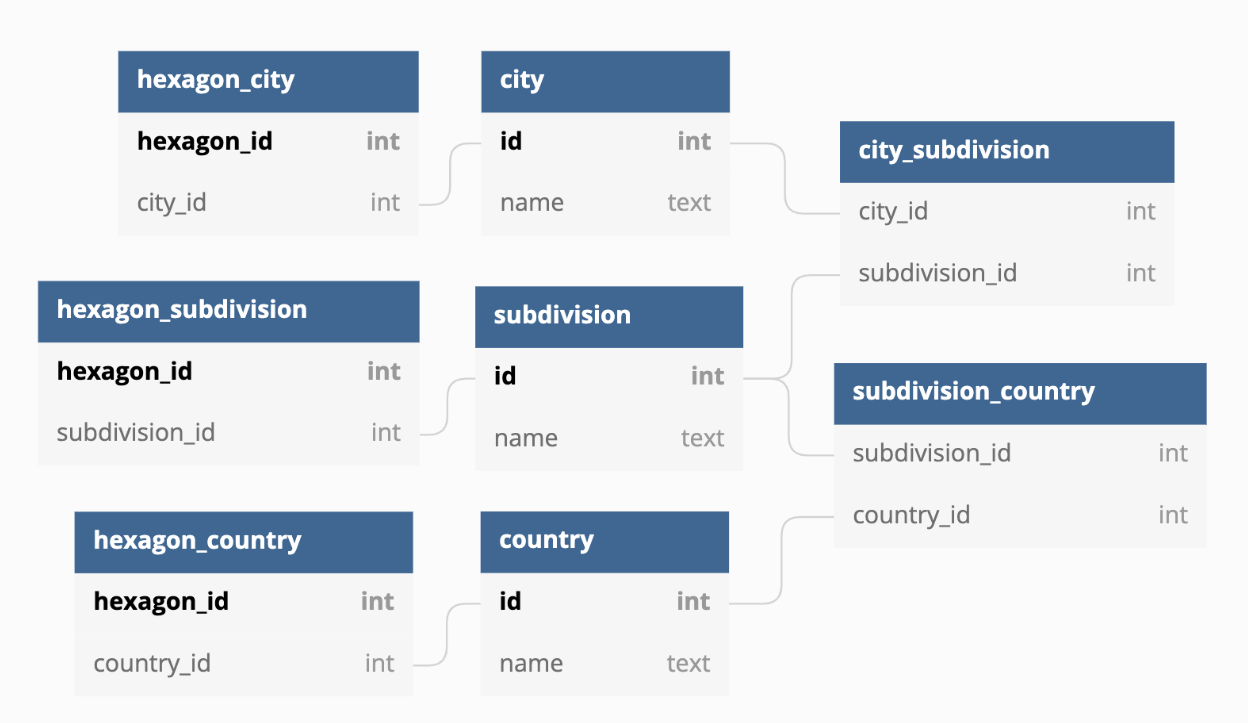

hexagon_city
hexagon_id int
city_id int
city
id int
name text
city_subdivision
city_id int
subdivision_id int
hexagon_subdivision
hexagon_id int
subdivision_id int
subdivision
id int
name text
subdivision_country
subdivision_id int
country_id int
hexagon_country
hexagon_id int
country_id int
country
id int
name text

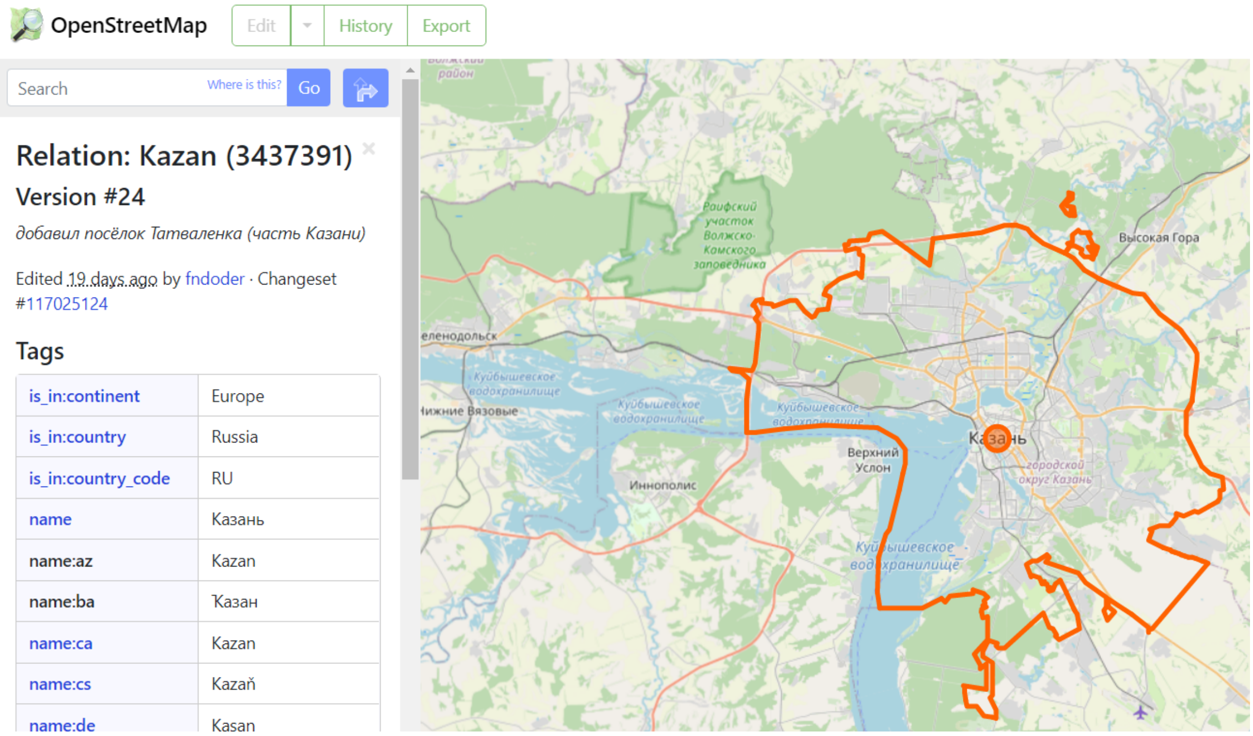
Fig. 3.4. Data model with separate tables (normalized).

These two approaches have their pros and cons. Storing additional data and relations in separate tables significantly reduces the occupied space in the database as the data are not duplicated. However, fetching the information related to POIs requires JOIN operation across multiple tables, which is more expensive than just retrieving a row from one table. On the other hand, denormalization improves the performance of SELECT queries at the price of database memory. The choice of the data model depends on the quality attributes of the software system. If the process of reverse geocoding is infrequent and not time-critical, the normalized database would satisfy the needs. However, if there are thousands of reverse geocoding requests per second, or if the software system is time-critical, the denormalized database would provide lower latency and higher throughput. I will use the denormalized model as the system should be highly efficient.

3.3 Dataset

The Database of Global Administrative Areas (GADM) provides spatial data of the whole world with six administrative subdivisions [19]. It includes geographic names and regional boundaries in the format of shapefiles which most GIS applications use. The database is available for academic and non-commercial use. Section 4 describes how to extract boundaries for all countries and their subdivisions from the dataset, find all corresponding H3 hexagons of the specified resolution and populate the database. Besides countries, GADM contains smaller divisions, such as regions and districts.

While countries and subdivisions are available in GADM, boundaries of cities are not easily accessible. Github repository `drei01/geojson-world-cities` contains 37,372 cities with boundaries in GeoJSON format. However, the data contain only the names of cities. As a result, it might be difficult to determine which country a city belongs to. For example, this dataset contains 11 different `SPRINGFIELD`s. Another option would be to use OpenStreetMap (OSM) database and extract GeoJSON for every desired city [20]. Fig. 3.5 shows the search result by the term “Kazan“. The result includes a list of tags and the boundaries of Kazan displayed on the map. However, retrieving boundaries for all cities worldwide in such a way is not trivial. Although it is possible to download the whole OSM dataset, which is 100 GB in a compressed archive, it is quite difficult to extract cities in the desired format (GeoJSON). SimpleMaps provides a World Cities Database [21] including around 41 thousand prominent cities with name, latitude, longitude, country, admin name, and population, but without boundaries. In this study I will not include cities in the reference dataset due to lack of a unified format.


OpenStreetMap
Edit
History
Export
Search
Where is this?
Go
Tags
волжский район
Раифский участок Волжско-Камского заповедника
Высокая Гора
еленодольск
Куйбышевское водохранилище
ижние Вязовые
Куйбышевское водохранилище
Куйбышевское водохранилище
Верхний Услон
Казань
городской округ Казань
Иннополис
Куйбышевское водохранилище

Relation: Kazan (3437391)

Version #24

добавил посёлок Тавваленка (часть Казани)

Edited 19 days ago by fndoder · Changeset #117025124

is_in:continent	Europe
is_in:country	Russia
is_in:country_code	RU
name	Казань
name:az	Kazan
name:ba	Ҡазан
name:ca	Kazan
name:cs	Kazaň
name:de	Kasan

Fig. 3.5. Search result from OpenStreetMap. (term: “Kazan“)

3.4 Database

The designed approach should be highly scalable, and so should the database. I compared two database management systems (DBMS): Apache Cassandra and Clickhouse.

Apache Cassandra is an open-source NoSQL distributed database with high scalability and availability [22]. Rows partition the data with one or more columns as a primary key. It is highly performant in both read and write operations. However, it does not support JOIN operations on tables. Taking into account Cassandra supports data compaction, it can easily handle a large amount of data. Cassandra is well suited for the denormalized data model. I will choose ScyllaDB as a primary database, which is compatible with Apache Cassandra but is more efficient.

Clickhouse describes itself as "an open-source, high-performance columnar OLAP database management system for real-time analytics using SQL" [23]. It is not as performant as Apache Cassandra in a single read or write operations, so the operations should be done in batches. Unlike Cassandra, it supports JOIN operations. Therefore, a user can store IDs, names, and other properties in a separate table, and join them with intermediate query execution results. Another advantage is functions for working with H3 indexes which are available out of the box. For example, `geoToH3` function converts a pair of coordinates into the ID of a hexagon with a specified resolution. Clickhouse is an optimal choice if it is already used to store analytical data. Adding new tables from the normalized data model will enable Clickhouse to aggregate and analyze geographical data by running more complex queries.

Chapter 4

Implementation

The implementation of the reverse geocoding algorithm consists of three parts:

1. processing the dataset;
2. populating the database;
3. querying the database.

The first two parts are depicted in Fig. 4.1.

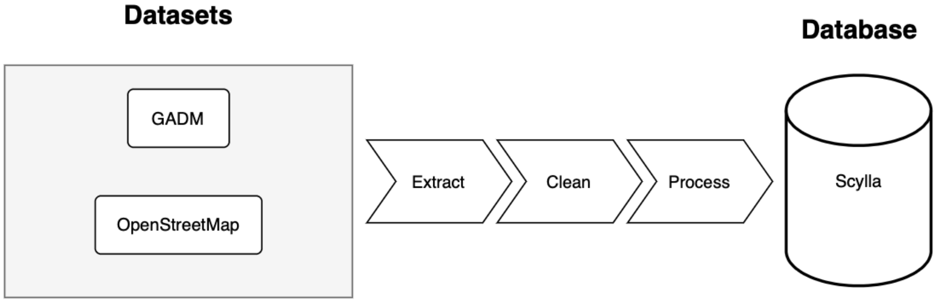

Datasets
GADM
OpenStreetMap
Extract
Clean
Process
Database
Scylla

Fig. 4.1. Data preparation for the reverse geocoding algorithm.

Sections 4.1, 4.2, and 4.3 describe three parts of the algorithm accordingly.

4.1 Processing dataset

4.1.1 Data extraction

GADM presents the data in several different formats [24]. A user can download the data by country or for the whole world. As an alternative, the dataset with six separate layers is available. The format of the files is either a single GeoPackage or multiple shapefiles. The study uses the latter because it is easier to process using Python.

Each of the six layers consists of:

- `cpg` file with the character sets used to display text in shapefiles in GIS software;
- `prj` file containing information about the coordinate system and projection;
- `shp` file, which is a shapefile describing geometry of objects;
- `dbf` file, which is a dBase database file.

Table I shows the format of dbf files.

TABLE I
Format of DBF files

(a) `gadm36_0.dbf` - information about countries

GID_0	NAME_0
ABW	Aruba
AFG	Afghanistan
AGO	Angola

(b) `gadm36_1.dbf` - information about country subdivisions.

GID_0	NAME_0	GID_1	NAME	VARNAME	TYPE_1	ENGTYPE_1	HASC_1
AFG	Afghanistan	AFG.1_1	Badakhshan	Badahšan	Velayat	Province	AF.BD
AFG	Afghanistan	AFG.2_1	Badghis	Badghes|Badghisat	Velayat	Province	AF.BG
AFG	Afghanistan	AFG.3_1	Baghlan	Baglan|Bughlan	Velayat	Province	AF.BL

The same information about countries and subdivisions is also stored in shapefiles. To read them, I use `Fiona` library, which is available in the Python Package Index (PyPI) [25]. `Fiona` represents objects stored in the shapefile as follows:

```
{'type': 'Feature', 'id': '0', 'properties': OrderedDict([('GID_0', 'ABW'),
↪  ('NAME_0', 'Aruba')]), 'geometry': {'type': 'Polygon', 'coordinates':
↪  [[(-69.97819519042969, 12.469861030578727), (-69.97846984863281,
↪  12.469861030578727), (-69.97846984863281, 12.47013854980463), ...}}
```

For countries, every object is a `Feature` that has an identifier (`id`), same properties as in `dbf` file (country code and name), and the boundaries in GeoJSON format. For subdivisions, it is identical but with a larger number of properties.

To store geographical objects internally, I created abstract class `GADMBaseModel`, which must implement class method `from_shapefile_object` to convert a dictionary into the instance of the model class.

```
class GADMBaseModel(ABC):
    @classmethod
    @abstractmethod
    def from_shapefile_object(cls: type[C], obj: Dict[str, Any]) -> C: ...
```

Then, I created subclasses of the base class for countries and subdivisions, respectively:

```
@define
class GADMCountry(GADMBaseModel):
    """
    Information about a country from GADM.
    (https://gadm.org/metadata.html)
    """

    id: int
    "Alternative unique identifier"

    name: str
    "Official name in Latin script"

    code: str
    "Preferred unique ID, starts with the ISO 3166-1 alpha-3 country code"

    geometry: list[Polygon]
    "List of polygons representing country boundaries"

    @classmethod
    def from_shapefile_object(cls, obj: Dict[str, Any]) -> "GADMCountry": ...

    def to_cql_model(self, hex_id: int) -> HexCountry: ...

@define
class GADMCountrySubdivision(GADMBaseModel):
    """
    Information about country subdivision (administrative unit) from GADM.
    (https://gadm.org/metadata.html)
    """

    id: int
    "Alternative unique identifier"

    name: str
    "Official name in Latin script"

    code: str
    "Preferred unique ID, starts with the ISO 3166-1 alpha-3 country code"

    geometry: list[Polygon]
    "List of polygons representing subdivision boundaries"

    other_names: list[str]
    "Alternative names in usage for the place"

    localized_names: list[str]
    "Official names in a non-Latin script"

    administrative_type: str
    "Administrative type in English"

    localized_administrative_type: str
    "Administrative type in local language"

    country_code: str
    "Preferred unique ID of the country"

    country_name: str
    "Official country name in Latin script"

    hasc_code: Optional[str]
    "HASC. A unique ID from Statoids"

    @classmethod
    def from_shapefile_object(cls, obj: Dict[str, Any]) -> "GADMCountrySubdivision": ...

    def to_cql_model(self, hex_id: int) -> HexCountrySubdivision: ...
```

To make classes more compact I use `attrs` library, which automatically generates the constructor and object protocols. Method `to_cql_model` will be used later for the database layer.

To represent data extractors for GADM dataset, I created abstract class `GADMBaseExtractor`, which implements an Iterator protocol. Extractors read files from the dataset and convert them to the internal model representation.

```
class GADMBaseExtractor(Generic[GADMModelType]):
    def __init__(self, path: Path) -> None:
        self.path = path

    @property
    @abstractmethod
    def layer_name(self) -> str:
        """Name of the layer from GADM shapefile."""

    @property
    @abstractmethod
    def model_class(self) -> type[GADMModelType]:
        """GADM Model specific to objects from the layer."""

    def __iter__(self) -> Iterator[GADMModelType]:
        with fiona.open(self.path, layer=self.layer_name) as src:
            for obj in src:
                yield self.model_class.from_shapefile_object(obj)
```

Subclassing the base class enables developers to add extractors for different layers of GADM dataset:

```
class GADMCountriesExtractor(GADMBaseExtractor[GADMCountry]):
    layer_name = "gadm36_0"
    model_class = GADMCountry


class GADMCountrySubdivisionsExtractor(GADMBaseExtractor[GADMCountrySubdivision]):
    layer_name = "gadm36_1"
    model_class = GADMCountrySubdivision
```

It is worth mentioning that the boundaries of geographical objects are not always represented by `Polygon`. If an area is split into several regions, it is represented as `Multipolygon`. For convenience, I convert every `MultiPolygon` to a list of Polygons.

4.1.2 Tiling areas with hexagons

The H3 Core Library implements different functions operating on cells of the H3 grid system [26]. Table II categorizes the functions by their purpose and typical usage.

TABLE II
H3 Core Library overview

Functions	**Use**
Indexing	Find the hexagon containing the specified point and calculate the center and boundaries of cells.
Inspection	Provide metadata about an H3 index (resolution and base cell).
Traversal	Find adjacent hexagons and calculate the distance in grid cells.
Hierarchy	Find the parent and children cells.
Regions	Convert polygonal areas to hexagons and vice versa.
Unidirectional edges	Encode edges between adjacent cells.
Miscellaneous	Describe the H3 grid system and provide helper functions.

Function `polyfill` from the library takes a Polygon in GeoJSON format and fills the region with hexagons of a specified resolution. If the boundaries are represented as MultiPolygon, they first must be converted into a list of Polygons. Then, each polygon is filled with hexagons. Finally, all the hexagons are gathered into the resulting set.

As an example, I filled the administrative boundaries of Kazan with H3 cells. Fig. 4.2 shows the visualization of regions and hexagons on the map using

Kepler.gl web application [27].

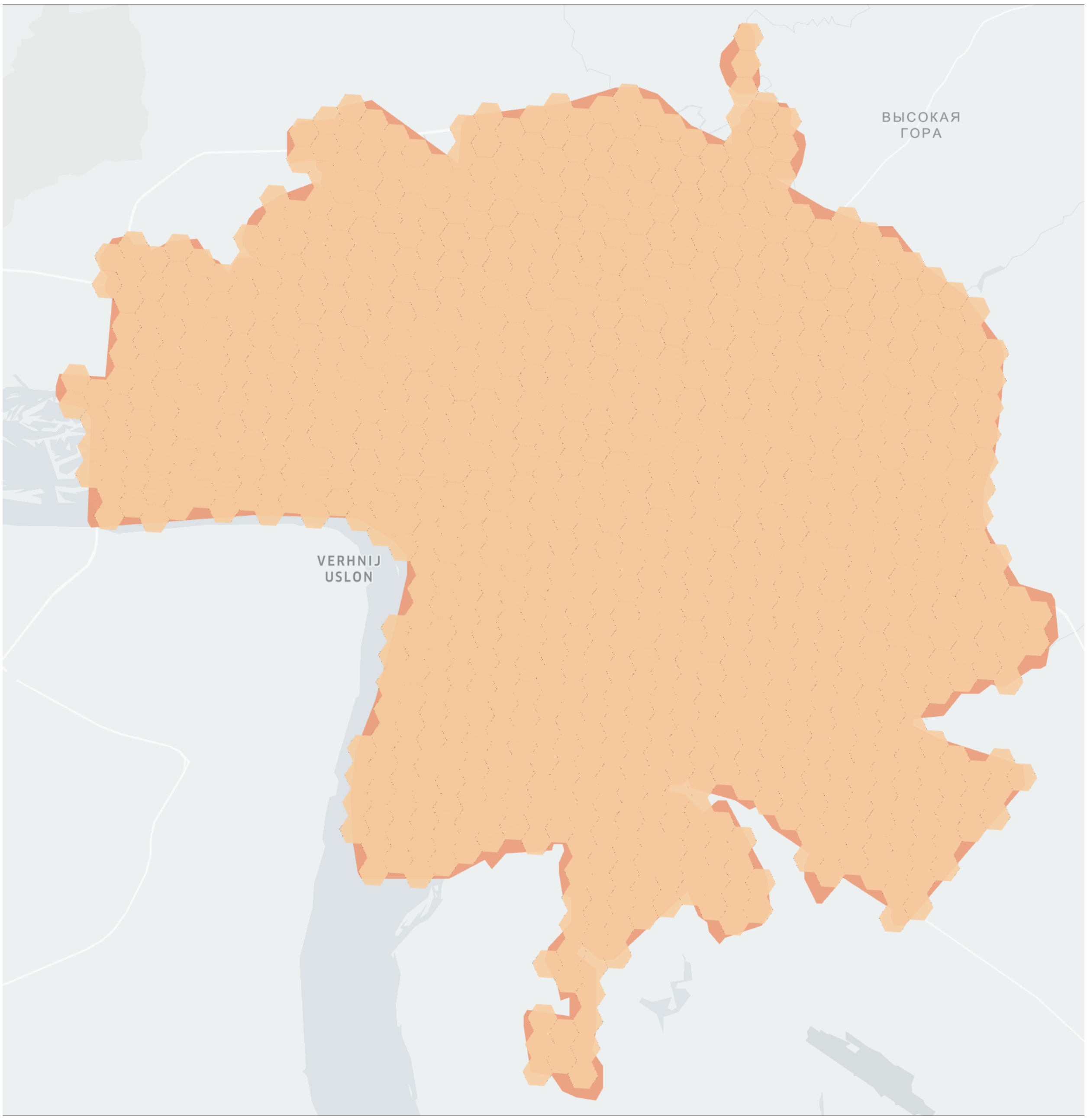


Fig. 4.2. Region of Kazan tiled with hexagons of resolution 7.

As shown in Fig. 4.3, some parts of the region are not tiled with hexagons at all. The hexagon is included in the resulting set if and only if the center lies inside the region. This leads to the following property: if polygons cover an area without overlap, cells will cover the same area without overlap. Countries are examples of such polygons - they are located next to each other without overlap, and there are too few gaps between them. Therefore, after tiling adjacent countries with hexagons, there will be no gaps between them.

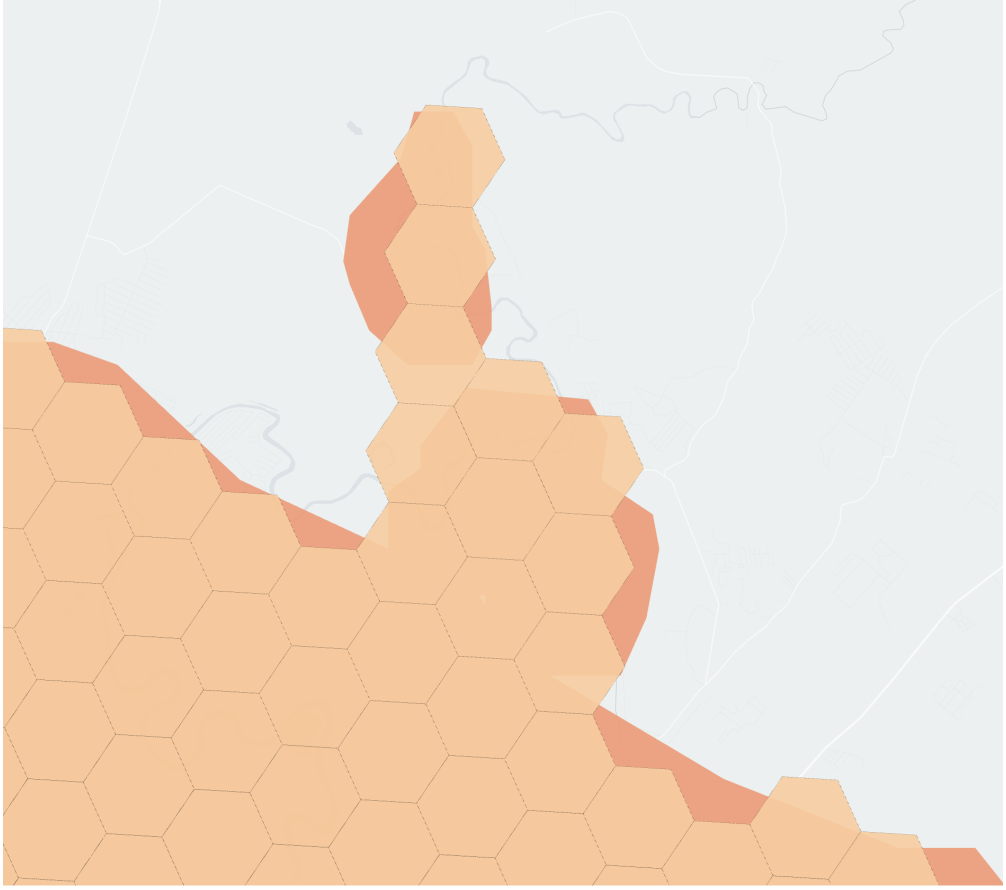


Fig. 4.3. A boundary of Kazan tiled with hexagons of resolution 7.

However, cities are rarely close to each other. Thus, some of the boundary areas are not covered by the hexagons. To minimize such areas, hexagons with higher resolutions can be used. Table III lists all the resolutions with corresponding hexagon sizes. There is a tradeoff between reverse geocoding precision and the computer resources the algorithm will require. The higher resolution, the more accurate reverse geocoding will be. However, it makes little sense to pick the highest resolution as the precision is also limited by the accuracy of datasets. With a higher resolution, the filling algorithm is more computationally expensive and requires more memory in the database.

TABLE III

H3 resolution table [18]

H3 Resolution	Average Hexagon Area (km2)	Average Hexagon Edge Length (km)	Number of unique indexes
0	4,250,546.8477000	1,107.712591000	122
1	607,220.9782429	418.676005500	842
2	86,745.8540347	158.244655800	5,882
3	12,392.2648621	59.810857940	41,162
4	1,770.3235517	22.606379400	288,122
5	252.9033645	8.544408276	2,016,842
6	36.1290521	3.229482772	14,117,882
7	5.1612932	1.220629759	98,825,162
8	0.7373276	0.461354684	691,776,122
9	0.1053325	0.174375668	4,842,432,842
10	0.0150475	0.065907807	33,897,029,882
11	0.0021496	0.024910561	237,279,209,162
12	0.0003071	0.009415526	1,660,954,464,122
13	0.0000439	0.003559893	11,626,681,248,842
14	0.0000063	0.001348575	81,386,768,741,882
15	0.0000009	0.000509713	569,707,381,193,162

One of the optimizations might be to use hexagons of different resolutions. The H3 Core Library includes function `compact`, which compresses the set of hexagons as well as possible. This means that smaller hexagons are converted into larger ones if possible. This can substantially decrease the number of INSERT operations to the database and save the memory. However, as demonstrated in 4.4, this also leads to the areas inside the regions which are not covered. If a point falls inside a gap, the reverse geocoding algorithm will fail. This is unacceptable for reverse geocoding, so I will cover regions by hexagons of the same resolution.

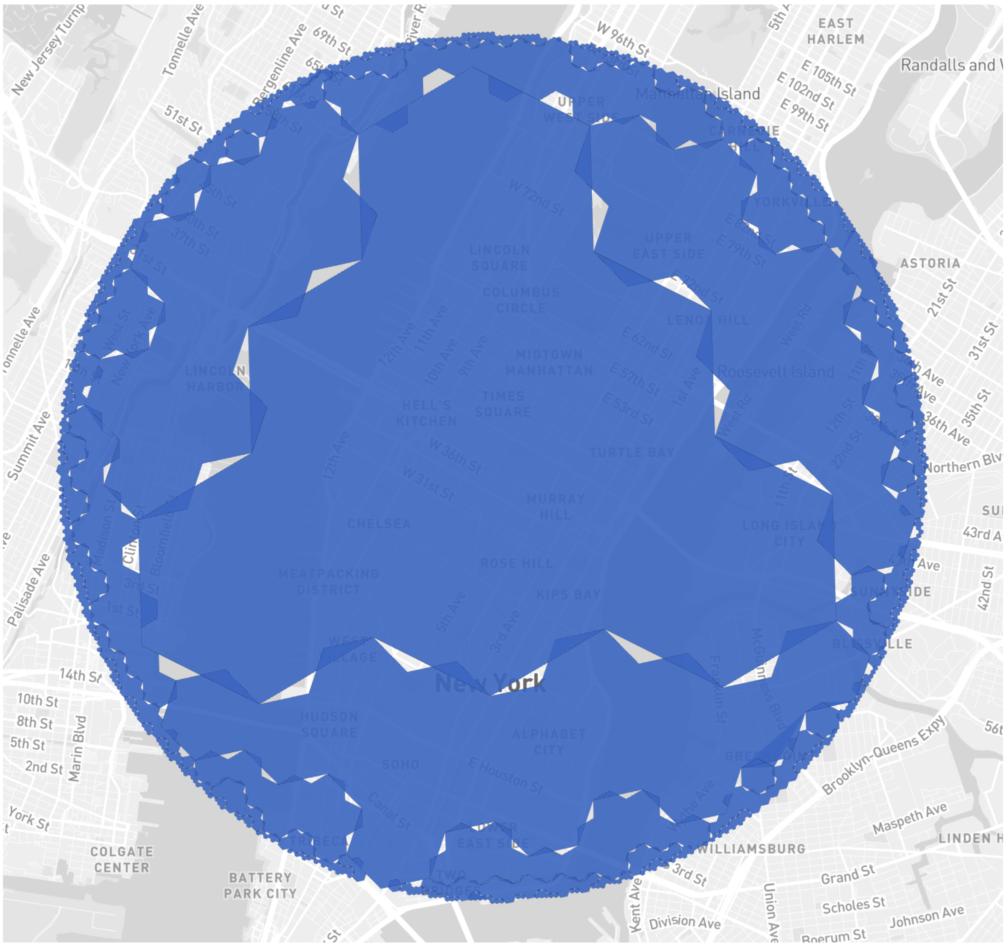
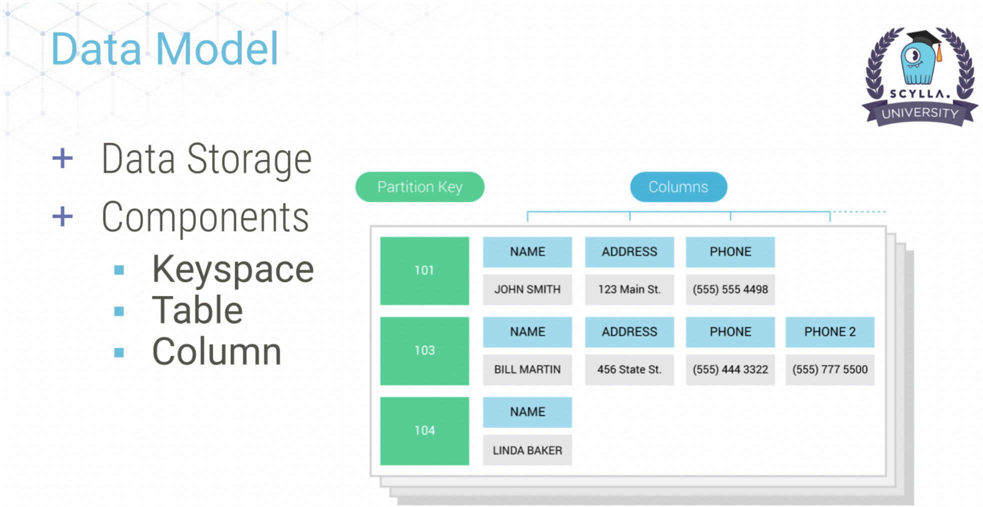


Fig. 4.4. Uncovered areas inside the region after the compaction.

4.2 Populating Database

Scylla stores all the data in a set of rows that form tables. Columns define the data structure in the table. Each column has a data type, such as `Integer` or `Text`. The data in tables are partitioned by the primary key which is made of one or more columns. Later, a user can query a specific row in a table using the primary key. The highest level of the data model is Keyspaces which contain tables.



Data Model

- + Data Storage
- + Components
  - Keyspace
  - Table
  - Column

101	NAME JOHN SMITH	ADDRESS 123 Main St.	PHONE (555) 555 4498	
103	NAME BILL MARTIN	ADDRESS 456 State St.	PHONE (555) 444 3322	PHONE 2 (555) 777 5500
104	NAME LINDA BAKER			

Fig. 4.5. Data model in Scylla.

Scylla uses The Cassandra Query Language (CQL) to create, read, update, and delete data. It also has a Python driver, which allows executing queries in Scylla using a convenient interface. Moreover, the driver supports Object-relational mapping (ORM), which converts Python classes to CQL tables.

To run Scylla server on the local machine, I used `scylladb/scylla` Docker image. Docker is a software tool that makes it easy to run applications on any operating system via virtualization. Firstly, I created a keyspace to store all project-related tables. The following command creates a new keyspace named `geo` with no data replication for testing purposes. As Scylla is highly scalable, it supports the data replication among multiple machines in the database cluster. In production-ready applications, these settings must be tuned according to the cluster specification.

```
CREATE KEYSPACE IF NOT EXISTS geo
```

```
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'};
```

Secondly, I created tables in the created keyspace. I will describe tables using Python classes:

```
class HexCountry(CQLModel):
    __table_name__ = "hex_countries"

    hex_id = columns.BigInt(primary_key=True)
    id = columns.Integer()
    name = columns.Text()
    code = columns.Text()


class HexCountrySubdivision(CQLModel):
    __table_name__ = "hex_subdivisions"

    hex_id = columns.BigInt(primary_key=True)
    id = columns.Integer()
    name = columns.Text()
    code = columns.Text()
    other_names = columns.Set(columns.Text)
    localized_names = columns.Set(columns.Text)
    administrative_type = columns.Text()
    localized_administrative_type = columns.Text()
    country_code = columns.Text()
    country_name = columns.Text()
    hasc_code = columns.Text()
```

The first column is set to be the primary key so that information about the countries and their subdivisions can be queried by the hexagon ID.

Unfortunately, the ORM does not provide a way to create or read multiple rows simultaneously. This is essential for the algorithm, so I implemented a connector class, which can execute multiple queries at the same time. On top of that, I created storage classes for countries and subdivisions which provides a convenient interface for the database.

4.3 Querying database

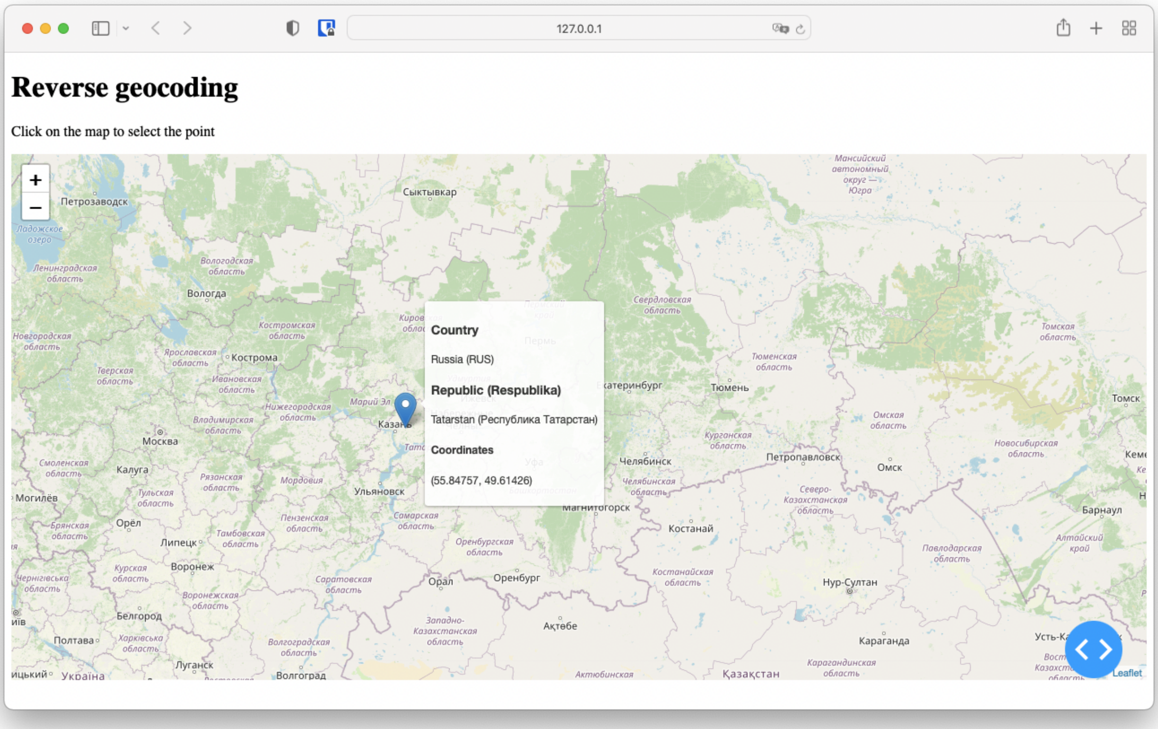
With multiple abstraction layers, the whole reverse geocoding algorithm fits into a few lines of code:

```python
def get_country_by_point(lat: float, lng: float) -> Optional[HexCountry]:
    hex_id = get_hexagon_by_point(lat, lng, settings.COUNTRIES_H3_RESOLUTION)
    return countries_storage.read(hex_id)
```

First, the coordinates are converted into H3 hexagon with a specified resolution. Then, a corresponding row with information about a country is retrieved from Scylla. For country subdivisions, the algorithm is identical.

4.4 Demo application

I developed a web application for demonstration of the reverse geocoding algorithm [28]. It represents an interactive map, which can be moved and zoomed in. When a user clicks at a specific point, the algorithm maps coordinates to a country and its subdivision. Then, the country and subdivision names are displayed above the map.


127.0.0.1
+
−
Leaflet
Петрозаводск
Сыктывкар
Вологда
Кострома
Москва
Калуга
Могилёв
Орёл
Липецк
Воронеж
Белгород
Полтава
Луганск
Волгоград
Украина
Казань
Ульяновск
Екатеринбург
Тюмень
Челябинск
Магнитогорск
Оренбург
Орал
Ақтөбе
Костанай
Петропавловск
Омск
Томск
Барнаул
Нур-Султан
Караганда
Қазақстан

Reverse geocoding

Click on the map to select the point

Country

Russia (RUS)

Republic (Respublika)

Tatarstan (Республика Татарстан)

Coordinates

(55.84757, 49.61426)

Fig. 4.6. Web application which uses the reverse geocoding algorithm.

Chapter 5

Evaluation and Discussion

5.1 Algorithm evaluation

5.1.1 Correctness

To test the correctness of the reverse geocoding algorithm, the set of coordinates should be supplied to an algorithm. Then, the locations produced by an algorithm should be compared with reference values. I will use dataset “Geolocation Data [Longitude Latitude]“ uploaded by You Sheng [29]. There 3 files in CSV (comma-separated values) format. Besides two coordinates columns (latitude and longitude), each of the files `countries.csv`, `states.csv`, and `cities.csv` include additional information related to countries, subdivisions, and cities accordingly. Tables IVa, IVb, and IVc describe the format of CSV files in detail.

TABLE IV
Format of CSV files

(a) `countries.csv`

id	name	iso3	iso2	numeric_code	region	subregion	latitude	longitude
1	Afghanistan	AFG	AF	004	Asia	Southern Asia	33.000000	65.000000
2	Aland Islands	ALA	AX	248	Europe	Northern Europe	60.116667	19.900000
3	Albania	ALB	AL	008	Europe	Southern Europe	41.000000	20.000000

(b) `states.csv`

id	name	country_id	country_code	country_name	state_code	latitude	longitude
3901	Badakhshan	1	AF	Afghanistan	BDS	36.7347725	70.8119953
3871	Badghis	1	AF	Afghanistan	BDG	35.1671339	63.7695384
3875	Baghlan	1	AF	Afghanistan	BGL	36.1789026	68.7453064

(c) `cities.csv`

id	name	state_id	state_code	state_name	country_code	country_name	latitude	longitude
52	Ashkāsham	3901	BDS	Badakhshan	AF	Afghanistan	36.68333	71.53333
68	Fayzabad	3901	BDS	Badakhshan	AF	Afghanistan	37.11664	70.58002
78	Jurm	3901	BDS	Badakhshan	AF	Afghanistan	36.86477	70.83421

First, I tested the reverse geocoding algorithm on all the coordinates from `countries.csv`. As the countries are queried from the database, they follow the format described in Section 4.2. The process of validating the result is straightforward - it is enough to compare ISO 3166-3 country codes. The dataset contains 250 countries and 5006 subdivisions. Table Va shows the accuracy of the algorithm for countries. The results are classified by:

- nulls - The algorithm failed to map these points giving no result.
- wrong - The algorithm gives the result different from desired.
- correct - The algorithm correctly identifies locations.

46 points from the source dataset were not mapped to any country. Most of these points specify location which are not in any country territory. For

example, coordinates of American Samoa (-14.3333, -170.0000) specify a point between 2 islands which is outside the territory, as depicted in Fig. 5.1. It is worth noting, some of the countries from the dataset do not constitute independent countries (i.e. Bouvet Island, French Polynesia). 6 points were probably misclassified because of partitioning errors and datasets discrepancy.

TABLE V

Accuracy of the reverse geocoding algorithm.

(a) countries

	number	**percentage**
total	250	
nulls	35	14%
wrong	5	2%
correct	210	84%

(b) subdivisions

	number	**percentage**
total	5006	
nulls	126	2.52%
wrong	2906	58.05%
correct	1974	39.43%

Testing the algorithm for subdivisions is almost the same as for countries. Nevertheless, the format of state codes in the database with locations is not uniform, so it is difficult to compare with subdivisions from GADM dataset. I used Jaro similarity index to calculate the similarity between two names. The names of subdivisions from different datasets were compared by names based on their similarity. The obtained results are shown in Table Vb.

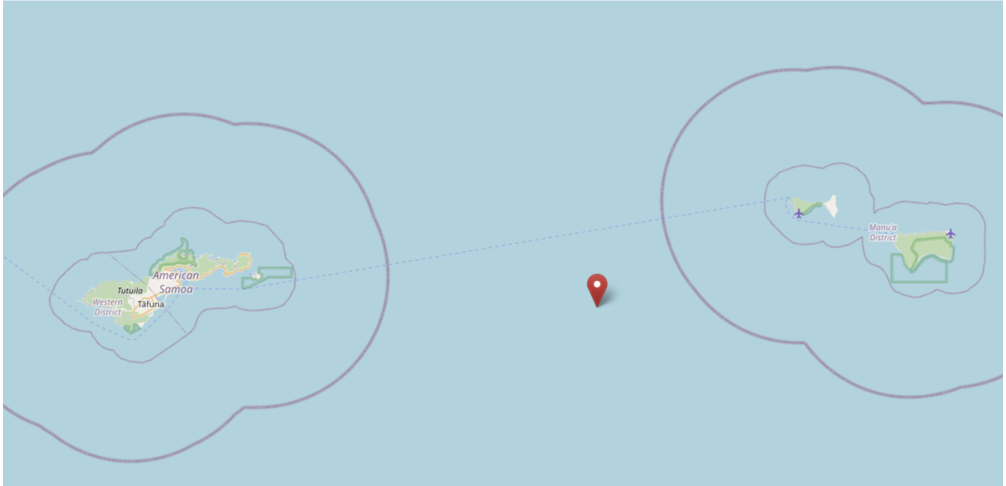


Fig. 5.1. The point is located between two islands of American Samoa.

5.1.2 Performance

I measured the time required to fill indexes for H3 resolutions from 0 to 7. The algorithm was run on MacBook Pro 2021 with Apple M1 Pro and 32 GB of RAM. Table VI shows the measurements of the time required to fill the database to run the reverse geocoding algorithm.

TABLE VI
Measurements of the time required to tile all countries and their subdivision by hexagons of different resolutions.

(a) countries

Resolution	Total number of unique indexes	Filling time, in seconds	Number of filled indexes
0	122	95.7	30
1	842	133.5	238
2	5,882	107.2	1,657
3	41,162	160.2	11,653
4	288,122	226	81,721
5	2,016,842	793	571,984
6	14,117,882	4084.1	4,003,622
7	98,825,162	25060.9	25,823,467

(b) country subdivisions

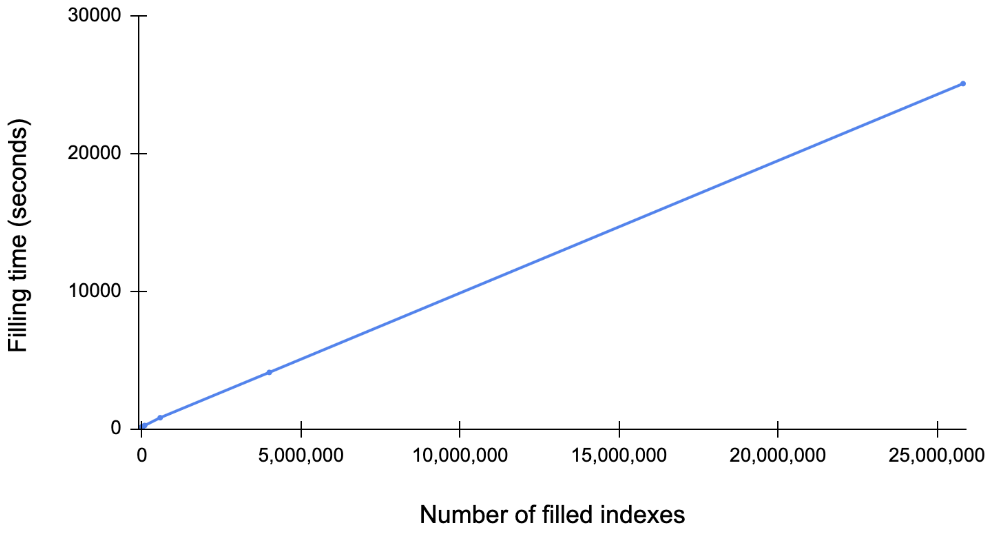
Resolution	Total number of unique indexes	Filling time, in seconds	Number of filled indexes
0	122	106.4	27
1	842	142	219
2	5,882	116.2	1,520
3	41,162	163.1	10,695
4	288,122	150.3	74,971
5	2,016,842	293	524,872
6	14,117,882	910.7	3,673,748
7	98,825,162	5506	25,716,536

Graphs in Fig 3.1 depict the filling time depending on the number of filled indexes. These charts are very similar to the graphs of linear functions, especially for resolutions of three and higher. Let *res* be resolution, *icount* -

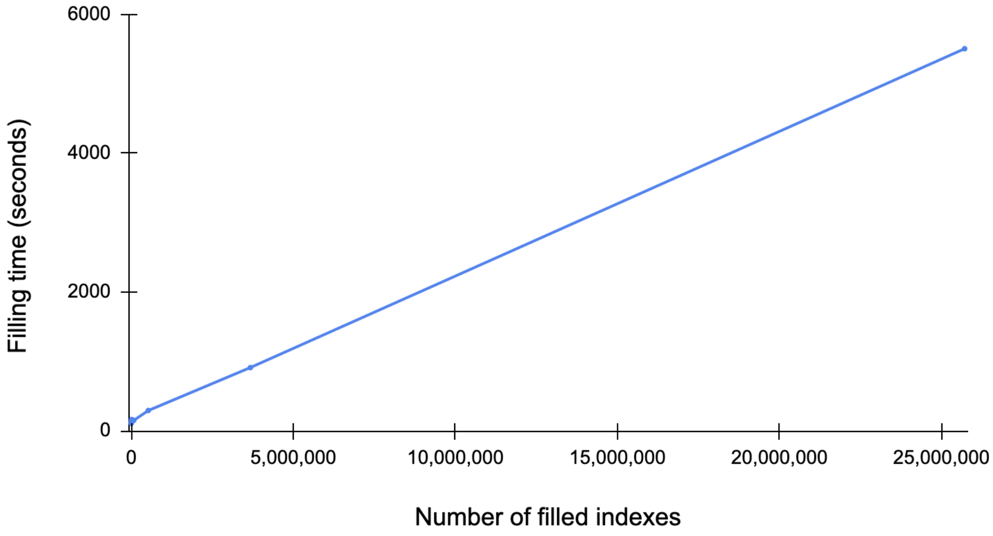
the number of filled indexes, and *time* - the filling time. Based on the graph, I assume $time = O(icount)$. As the resolution is incremented by one, the number of unique indexes is increased by 7 times. Therefore, $icount \sim 7^{res}$ and $time(res) = O(7^{res})$. The filling algorithm has exponential time complexity with respect to resolution.

Fig. 5.2. Dependency of the filling time on the number of filled indexes.

(a) countries



(b) subdivisions



To measure the performance of the reverse geocoding algorithm, I generated coordinates of one million points. It took 346.82 seconds for the algorithm to determine countries and subdivisions for these points. On average, the algorithm processes 2883.35 points per second. The performance of the querying highly depends on the database.

5.1.3 Conclusion

The accuracy of the algorithm for countries is higher than for subdivisions because countries are presented in a unified format, unlike subdivisions. It is difficult to match country subdivisions from different datasets because of different names and codes. The algorithm was tested for H3 indexes of resolution 7. It is worth setting a higher resolution as long as the hexagon edge length is within the margin of error in the dataset.

The performance was tested on a local machine in Docker. The number of points which can be reverse-geocoded highly depends on the performance of the database in read operations. If the database was on a dedicated server, the throughput would be different.

Chapter 6

Conclusion

In this work, I described how to implement the algorithm for reverse geocoding using Uber H3. The algorithm uses a software system, which consists of a client library and a database. The client library includes functions, which process the data and interact with the database. Uber H3 was used to map coordinates into integer indexes for the database and vice versa.

The main criteria for the algorithm were efficiency and scalability. Both of them were achieved by Scylla, which provides high performance and can be scaled linearly by adding more nodes and data centers. Both performance and accuracy of the algorithm were tested on a local computer. The throughput of several thousand points per second was achieved with a single-node cluster of Scylla.

Limitations

The implementation of the reverse geocoding algorithm can only convert coordinates into a corresponding country and its subdivision. Commonly, it is substantial to map points into complete postal addresses. In such cases, it makes more sense to use conventional or online reverse geocoding. Nevertheless, the implementation based on Uber H3 is more efficient for aggregations of countries and subdivisions among thousands of geographical locations.

Further work

In section 4.2 I mentioned that I did not implement reverse geocoding for cities because I could not find any dataset with a unified format except OpenStreetMap. Future studies could examine how to extract city boundaries from OSM dataset in GeoJSON format, and implement the reverse geocoding algorithm by analogy with countries.

In section 4.1.2 I described a possible optimization for the algorithm which might significantly reduce the size of the reference dataset. However, the compaction algorithm produces gaps between adjacent hexagons of different resolutions. If there was no such problem, it would be possible to look up the row in the database by indices of hexagons from low to high resolutions.

As demonstrated in Section 5.1.2, tiling a region by hexagons is computationally expensive. The algorithm execution time is proportional to the number of hexagons in the area. Future research might improve the algorithm to get a better time complexity than exponential. In addition, it might examine the source code of H3 Core Library.

Bibliography cited

[1] D. Li, "1.08 - geocoding and reverse geocoding," in *Comprehensive Geographic Information Systems*, B. Huang, Ed., Oxford: Elsevier, 2018, pp. 95–109, ISBN: 978-0-12-804793-4. DOI: `https://doi.org/10.1016/B978-0-12-409548-9.09593-2`. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/B9780124095489095932`.

[2] [Online]. Available: `https://developers.google.com/maps/documentation/geocoding/overview`.

[3] I. Brodsky, *H3: Uber's hexagonal hierarchical spatial index*, May 2019. [Online]. Available: `https://eng.uber.com/h3/`.

[4] G. Rushton, M. Armstrong, J. Gittler, B. Greene, C. Pavlik, M. West, and D. Zimmerman, *Geocoding health data: The use of geographic codes in cancer prevention and control, research and practice.* Jan. 2007, pp. 1–249.

[5] M. Cockburn, "The effect of administrative boundaries and geocoding error on cancer rates in california," *Spatial and spatio-temporal epidemiology*, vol. 3, pp. 39–54, Apr. 2012. DOI: `10.1016/j.sste.2012.02.005`.

[6] G. Rushton, M. P. Armstrong, J. Gittler, B. R. Greene, C. E. Pavlik, M. M. West, and D. L. Zimmerman, "Geocoding in cancer research: A

review," *American Journal of Preventive Medicine*, vol. 30, no. 2, Supplement, S16–S24, 2006, ISSN: 0749-3797. DOI: `https://doi.org/10.1016/j.amepre.2005.09.011`. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S0749379705003624`.

[7] S. Jünger, S. Schweers, and P. Siegers, "Geocoding and spatial linking of survey data: An introduction for social scientists," Feb. 2017. DOI: `10.21241/ssoar.52316`.

[8] S. Eckman and N. English, "Geocoding to create survey frames," *Survey Practice*, vol. 5, pp. 1–8, Dec. 2012. DOI: `10.29115/SP-2012-0026`.

[9] M. Andresen, S. Linning, and N. Malleson, "Crime at places and spatial concentrations: Exploring the spatial stability of property crime in vancouver bc, 2003–2013," *Journal of Quantitative Criminology*, vol. 33, pp. 255–275, Jun. 2017. DOI: `10.1007/s10940-016-9295-8`.

[10] Informatica LLC, *Improve business operations with geocoding*, 2021. [Online]. Available: `https://www.informatica.com/resources/articles/improve-business-operations-with-geocoding` (visited on 10/22/2021).

[11] B. Ramella. (2019). "Geocoding 101: What is geocoding and how does it work?" [Online]. Available: `https://learn.g2.com/geocoding` (visited on 10/22/2021).

[12] J. Wilson and C. Knoblock, "From text to geographic coordinates: The current state of geocoding," *Urisa Journal*, vol. 19, pp. 33–46, Jan. 2007.

[13] D. Asavasuthirakul and H. Karimi, "Comparative evaluation and analysis of online geocoding services," *International Journal of Geographical*

Information Science, vol. 24, pp. 1081–1100, Jun. 2010. DOI: `10.1080/13658810903289478`.

[14] K. Sahr, D. White, and A. J. Kimerling, “Geodesic discrete global grid systems,” *Cartography and Geographic Information Science*, vol. 30, no. 2, pp. 121–134, 2003. DOI: `10.1559/152304003100011090`.

[15] J. Bousquin, “Discrete global grid systems as scalable geospatial frameworks for characterizing coastal environments,” *Environmental Modelling and Software*, vol. 146, p. 105 210, 2021, ISSN: 1364-8152. DOI: `https://doi.org/10.1016/j.envsoft.2021.105210`. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S1364815221002528`.

[16] *S2 geometry*. [Online]. Available: `https://s2geometry.io/`.

[17] *S2*. [Online]. Available: `https://h3geo.org/docs/comparisons/s2`.

[18] *Table of cell areas for h3 resolutions*. [Online]. Available: `https://h3geo.org/docs/core-library/restable/`.

[19] *Gadm maps and data*. [Online]. Available: `https://gadm.org/`.

[20] *Openstreetmap*. [Online]. Available: `https://www.openstreetmap.org/`.

[21] *World cities database*. [Online]. Available: `https://simplemaps.com/data/world-cities`.

[22] *Open source nosql database*. [Online]. Available: `https://cassandra.apache.org/_/index.html`.

[23] *Clickhouse dbms*. [Online]. Available: `https://clickhouse.com/`.

[24] *Download gadm data*. [Online]. Available: `https://gadm.org/download_world.html`.

[25] *Fiona.* [Online]. Available: `https://pypi.org/project/Fiona/`.

[26] *Indexing functions.* [Online]. Available: `https://h3geo.org/docs/api/indexing`.

[27] *Large-scale webgl-powered geospatial data visualization tool.* [Online]. Available: `https://kepler.gl/`.

[28] a1d4r, *Reverse geocoding algorithm based on uber h3*, `https://github.com/a1d4r/reverse-geocoding-uber-h3`, 2022.

[29] Y. Sheng, *Geolocation data [longitude latitude]*, Mar. 2022. [Online]. Available: `https://www.kaggle.com/datasets/liewyousheng/geolocation/code`.